
AUTO SKILL: EXPERIENCE-DRIVEN LIFELONG LEARNING VIA SKILL SELF-EVOLUTION

Yutao Yang^{1,*}, Junsong Li^{1,*}, Qianjun Pan^{1,*}, Bihao Zhan¹, Yuxuan Cai¹, Lin Du¹, Jie Zhou^{1,2,†}, Kai Chen^{2,†},
Qin Chen¹, Xin Li², Bo Zhang², Liang He¹

¹ School of Computer Science and Technology, East China Normal University, ² Shanghai AI Laboratory
{jzhou, qchen, lhe}@cs.ecnu.edu.cn,
<https://github.com/ECNU-ICALK/AutoSkill>

ABSTRACT

In practical LLM applications, users repeatedly express stable preferences and requirements—such as reducing hallucinations, following institutional writing conventions, or avoiding overly technical wording—yet such interaction experience is seldom consolidated into reusable knowledge. Consequently, LLM agents often fail to accumulate personalized capabilities across sessions. We present AutoSkill, an experience-driven lifelong learning framework that enables LLM agents to automatically derive, maintain, and reuse skills from dialogue and interaction traces.

AutoSkill abstracts skills from user experience, supports their continual self-evolution, and dynamically injects relevant skills into future requests without retraining the underlying model. Designed as a model-agnostic plug-in layer, it is compatible with existing LLMs and introduces a standardized skill representation for sharing and transfer across agents, users, and tasks. In this way, AutoSkill turns ephemeral interaction experience into explicit, reusable, and composable capabilities.

This paper describes the motivation, architecture, skill lifecycle, and implementation of AutoSkill, and positions it with respect to prior work on memory, retrieval, personalization, and agentic systems. AutoSkill highlights a practical and scalable path toward lifelong personalized agents and personal digital surrogates.

Keywords Skill · Experience-Driven Lifelong Learning · Self-evolving

1 Introduction

Large language models [1, 2, 3] have enabled a new generation of interactive agents for writing assistance, planning, coding, decision support, and tool use [4, 5, 6, 7]. As these systems move from controlled benchmarks to real world deployment, a recurring pattern becomes increasingly visible: users repeatedly restate stable preferences and operating requirements across sessions. For example, a user may consistently ask the agent to avoid hallucinations, follow an official writing style, reduce technical jargon, or adhere to a preferred workflow. Recent work on memory enhanced agents and long horizon conversational settings has highlighted the importance of preserving user specific information over time [8, 9, 10, 11, 12]. However, in current practice, such repeated interaction experience is rarely transformed into reusable capability. As a result, user habits and task specific expectations often need to be reestablished from scratch in each new session.

This limitation reveals a broader challenge for personalized language agents. Existing approaches provide only partial solutions. Parameter updating and self evolution methods can improve model behavior through self reflection, feedback driven optimization, or self training [13, 14, 15, 16], but they are often costly or difficult to control in settings that require frequent and fine grained personalization. Memory based approaches preserve facts, preferences, or prior dialogue content through retrieval and long term storage [17, 8, 9, 12, 18, 19], yet they usually treat past interaction as text to be retrieved rather than behavior to be operationalized. Agent frameworks and skill learning methods have

*These authors contributed equally to this work.

†Corresponding Authors

demonstrated the value of reusable strategies for reasoning, tool use, and task execution [4, 5, 20, 21], but in many cases those skills remain implicit in prompts, trajectories, or policies. What is still missing is a mechanism that can convert recurring interaction experience into explicit, reusable, and maintainable skills.

In this paper, we present **AUTO SKILL**, an experience driven lifelong learning framework for large language model agents. The central idea of AutoSkill is to treat repeated interaction experience not merely as memory, but as a source of skill formation. Instead of storing only dialogue snippets or preference records, AutoSkill abstracts reusable behaviors from user interactions and crystallizes them into explicit skill artifacts. These artifacts capture behavioral patterns such as stylistic constraints, response strategies, tool use procedures, and domain specific operating conventions. Because they are represented in a structured form, skills can be inspected, edited, merged, versioned, and reused across sessions.

AutoSkill supports a full skill lifecycle. It identifies candidate skills from dialogue and interaction events, summarizes them into standardized `SKILL.md` artifacts, updates them through iterative refinement, and injects relevant skills into future requests at inference time. This design enables continual capability accumulation without retraining the underlying model. It also provides a practical interface for human oversight, since developers and users can directly inspect and revise the resulting skills. In this way, AutoSkill bridges short term interaction experience and long term capability development, moving language agents closer to the goal of becoming personal digital surrogates that reflect stable user habits, preferences, and working styles.

Beyond practical personalization, AutoSkill contributes a distinct perspective on lifelong learning for language agents. It shifts the unit of accumulation from raw memory records to explicit behavioral knowledge, and it frames agent improvement as a process of skill extraction, maintenance, and reuse. This perspective is important for both research and deployment. From a research standpoint, it offers a concrete representation for studying how interaction experience can become reusable capability. From a system standpoint, it provides a plug in layer that can work with existing language models and agent frameworks, while supporting skill sharing and transfer across tasks and users.

The main contributions of this paper are as follows:

- We formulate the problem of transforming interaction experience into explicit reusable skills for personalized large language model agents, and we introduce AutoSkill as a framework for this setting.
- We propose a skill lifecycle that covers skill extraction, structured representation, iterative refinement, retrieval, and reuse, enabling continual skill evolution without modifying base model parameters.
- We design skills as editable and versioned artifacts, which improves transparency, controllability, and long term maintainability compared with implicit memory or policy based approaches.
- We implement AutoSkill as an open source and deployable system that supports integration with existing large language models and agent pipelines, providing a practical path toward lifelong personalized agents.

2 Related Work

We organize related work into four research threads: lifelong learning from experience, self evolution for large language models, long term memory for language agents, and skill learning for reasoning and acting. AutoSkill is related to all four directions, but it is distinguished by its emphasis on explicit skill artifacts, human editable representations, and lifecycle management for skill extraction, revision, retrieval, and reuse.

2.1 Experience Driven Lifelong Learning

Experience driven lifelong learning studies how agents accumulate reusable knowledge, strategies, or policies from ongoing interactions, so that experience obtained in one setting can support performance in future tasks. Core questions in this line of work include when knowledge should be extracted from interaction history, what should be retained as reusable capability, and how noise accumulation and forgetting can be controlled over long horizons. Experience driven Lifelong Learning (ELL) formalizes these goals and introduces benchmarks such as `StuLife` for evaluating self evolving agents in long horizon environments [22]. Related surveys further organize the research landscape around the perception, memory, and action pipeline of lifelong LLM agents [23]. AutoSkill shares the goal of continual capability accumulation from real interactions, but differs in how experience is represented and maintained. Instead of keeping knowledge as latent memory or implicit policy adaptation, AutoSkill crystallizes reusable capabilities into explicit `SKILL.md` artifacts with versioned evolution. This design improves interpretability, supports manual inspection and revision, and makes sustained alignment with user preferences easier to achieve.

2.2 Self Evolution for Large Language Models

Self evolution methods aim to improve model behavior through self reflection, iterative rewriting, feedback driven refinement, or autonomous data construction. Representative work includes SELF, which introduces self evolution with language feedback [13]; *Large Language Models Can Self Improve*, which studies self training with unlabeled data [14]; and Recursive Introspection (RISE), which enables models to revise prior attempts through repeated introspection [15]. Self Evolving Curriculum (SEC) further explores automated curriculum construction for reasoning tasks [24], while recent surveys summarize the broader landscape of self evolving LLM systems [25]. Uncertainty enhanced preference optimization (UPO) is another representative approach, where model policies are improved through reliable feedback sampling [16]. AutoSkill is complementary to this line of research. It does not update model parameters or rely on implicit policy drift. Instead, it externalizes reusable behaviors into structured skill artifacts and supports their evolution through explicit revision, merging, and version control. This makes the improvement process more transparent and controllable, especially in scenarios where user preferences and working styles must remain stable across sessions.

2.3 Long Term Memory for Language Agents

Retrieval augmented generation (RAG) improves factuality and traceability by injecting retrieved external knowledge into the generation process [17]. Retrieval augmented pretraining and retrieval enhanced language models, including REALM [26] and RETRO [27], extend this idea by coupling parametric models with large non parametric memory. Dense and unsupervised retrieval methods such as DPR [28] and Contriever [29], as well as late interaction retrievers such as ColBERT [30], improve retrieval quality and efficiency. Fusion based methods including FiD and Atlas further demonstrate the effectiveness of retrieval augmented reasoning for question answering [31, 32]. Other approaches, such as kNN LM, extend model recall through nearest neighbor retrieval in representation space [33]. Beyond factual retrieval, memory oriented systems introduce mechanisms for long term storage and management across sessions, including MemoryBank [8], MemGPT [9], and generative agent architectures that organize episodic memories and reflections for planning [7]. Benchmarks such as LoCoMo [10], LongMemEval [11], and incremental multi turn memory evaluation [34] assess long horizon memory in conversational agents. More recent frameworks, including Mem0 [12], A MEM [18], and MemInsight [19], together with corresponding surveys [35], further systematize memory mechanisms for LLM based agents. AutoSkill builds on the insight that retrieval can reactivate useful past experience, but it moves beyond conventional memory systems by lifting memory from text records to behavior units. Through explicit skill abstraction, retrieval, and maintenance, AutoSkill is better suited for preserving stable preferences, stylistic constraints, and recurring workflows that are difficult to represent as raw text snippets alone.

2.4 Skill Learning for Reasoning and Acting Agents

Skill learning for LLM agents concerns the acquisition of reusable reasoning patterns, tool use procedures, and action strategies. Methods for agentic reasoning and decision making, such as ReAct [4], show that interleaving reasoning and acting can improve tool interactive problem solving. Tool use oriented approaches, including Toolformer [5], ART [36], ToolAlpaca [37], and Gorilla [6], further show that language models can learn to invoke external tools and APIs in increasingly general settings. Related benchmarks and datasets, such as API Bank [38] and ToolBench or ToolLLM [39], provide evaluation settings for tool use competence. In embodied and open ended environments, systems such as Voyager highlight the value of compositional skill libraries for continual exploration and reuse [20]. Reflexion also uses verbal feedback and memory updates to improve future decisions [21]. A range of agent benchmarks and environments, including WebShop [40], ALFWorld [41], WebArena [42], and AgentBench [43], further stress long horizon execution, planning, and skill generalization. However, in most existing approaches, skills remain implicit in prompts, trajectories, or latent policies, and therefore lack a unified mechanism for inspection, editing, transfer, and long term maintenance. AutoSkill addresses this limitation by treating skills as first class artifacts that can be extracted from interaction experience, edited by users or developers, merged across iterations, versioned over time, and dynamically injected into future tasks. This explicit extraction and maintenance loop is central to AutoSkill and enables sustained skill evolution in a controllable manner.

3 Method

We propose a training-free lifelong learning framework that improves dialogue quality through explicit skill self-evolution rather than parameter updates. The core idea is to externalize reusable task-solving patterns as versioned skills, retrieve them for future responses, and continuously refine them with newly observed user interactions. As shown in Figure 1, the framework consists of two coupled loops: *skill-enhanced response generation* and *skill evolution*. The former retrieves useful skills to assist the current response, while the latter updates the skill bank based on newly observed dialogue turns.

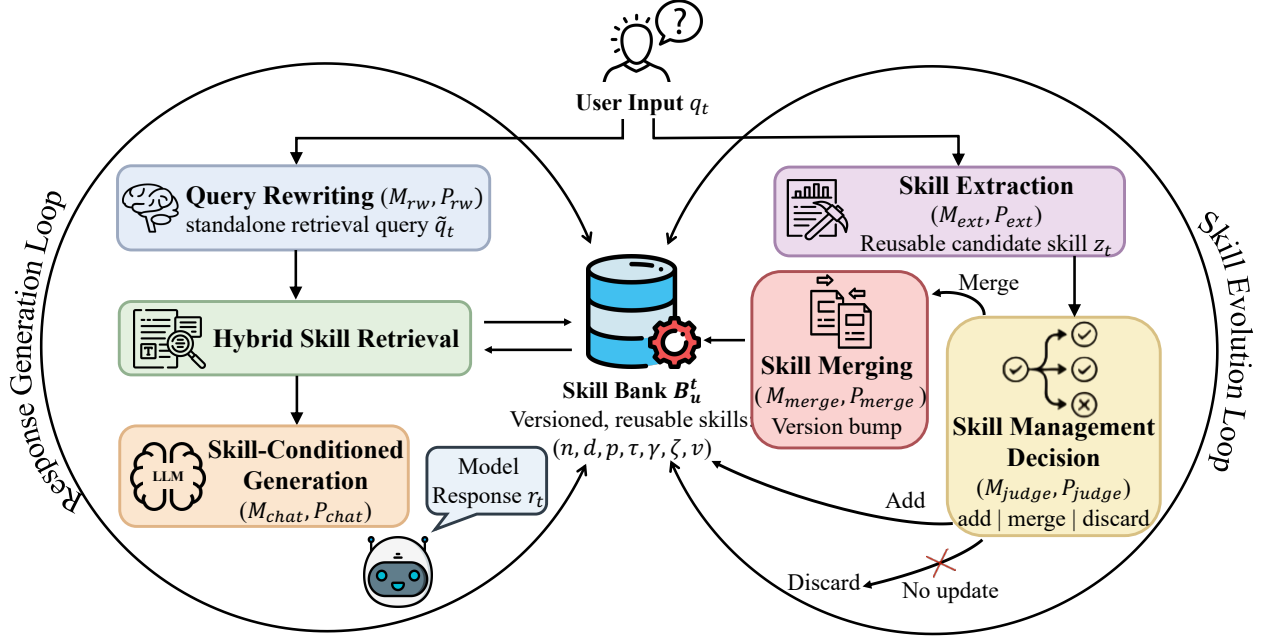


Figure 1: The Framework for our AutoSkill, which is composed of two tightly coupled processes. The right loop, *skill evolution*, transforms interaction experience into explicit skills through extraction and maintenance. The left loop, *skill-enhanced response generation*, uses the current skill bank to support response generation via query rewriting, skill retrieval, and context injection. In this way, the system continually improves through explicit memory growth rather than through model fine-tuning.

3.1 Problem Definition

For a user u , we denote the complete dialogue history as

$$\mathcal{X}_u = \{x_1, x_2, \dots, x_T\}, \quad x_t = (q_t, r_t),$$

where q_t is the user query at turn t and r_t is the model response. A skill bank $\mathcal{B}_u^t$ is maintained for user u after turn t . Each skill is represented as

$$s = (n, d, p, \tau, \gamma, \xi, v),$$

where n is the skill name, d is the description, p is the executable instruction prompt, τ is the trigger set, γ is the tag set, ξ is the example set, and v is the version.

Our method is **training-free**: no model parameters are updated during deployment. Instead, the system is implemented with five prompt-driven modules: a query rewriting model, a dialogue response model, a skill extraction model, a skill management judge, a skill merge model, and an embedding model for skill vectorization. Given the current query q_t and dialogue history, the system retrieves relevant skills from $\mathcal{B}_u^t$ to generate the response r_t ; meanwhile, it uses the *user-side* interaction signal to update the skill bank. Importantly, the skill extraction stage only uses user queries rather than model responses, i.e., it learns from $\{q_1, \dots, q_t\}$ but does not use r_t as extraction evidence.

3.2 Prompt-Driven Modular Architecture

All functional modules in our framework are realized by task-specific prompts rather than specialized training. Let

$$\mathcal{P} = \{P_{rw}, P_{chat}, P_{ext}, P_{judge}, P_{merge}\}$$

denote the prompt set for query rewriting, dialogue generation, skill extraction, skill management decision, and skill merging, respectively. Each module is instantiated by pairing a general-purpose LLM with its corresponding prompt. Therefore, our framework can be viewed as a modular inference-time composition:

$$\mathcal{M} = \{M_{rw}, M_{chat}, M_{ext}, M_{judge}, M_{merge}, M_{emb}\},$$

where M_{emb} is the embedding model used for dense vector retrieval.

This design has two advantages. First, different modules can share the same backbone LLM while serving different roles through different prompts. Second, the whole system remains highly flexible: replacing the response model, extraction model, or embedding model does not require retraining the framework itself.

3.3 Skill-Enhanced Response Generation

Given the current user query q_t and recent dialogue history $h_t \subset \mathcal{X}_u$, the system first rewrites the query into a retrieval-oriented form by a dedicated LLM prompt:

$$\tilde{q}_t = M_{\text{rw}}(P_{\text{rw}}, q_t, h_t).$$

The purpose of query rewriting is to resolve context dependence, preserve the current task anchor, and expose retrieval-critical constraints such as format, style, structure, or domain requirements.

P_{rw} : Query Rewriting

Role. Retrieval query rewriter.

Goal. Rewrite the current user input into exactly one concise, standalone search query for skill retrieval.

Core rules.

- First judge whether the current turn is a continuation of the same task or a topic switch.
- For continuation, preserve the existing *topic anchor* and append only the new constraints.
- For topic switch, replace the previous anchor with the new task/topic.
- If the current turn contains only style/format constraints, inherit the missing task anchor from recent history.
- Resolve references such as “it”, “this”, and “the above”.
- Keep only retrieval-relevant constraints (e.g., format, audience, quality requirements, banned structures).
- Avoid generic process words without a concrete task/topic anchor.

Output. One-line rewritten query, in the same language as the user.

3.3.1 Hybrid Skill Retrieval

For each skill $s \in \mathcal{B}_u^t$, we compute both a dense semantic relevance score and a lexical BM25 relevance score:

$$d(q_t, s) = \text{sim}(M_{\text{emb}}(\tilde{q}_t), M_{\text{emb}}(s)),$$

$$b(q_t, s) = \text{BM25}(\tilde{q}_t, s).$$

Since these two scores lie on different scales, we normalize them into $[0, 1]$ and combine them by weighted summation:

$$\text{Rel}(q_t, s) = \lambda \hat{d}(q_t, s) + (1 - \lambda) \hat{b}(q_t, s),$$

where $\lambda \in [0, 1]$ controls the trade-off between dense semantic matching and lexical exact matching.

We then rank all skills by $\text{Rel}(q_t, s)$ and keep only the top- K candidates whose score exceeds a predefined threshold η :

$$\mathcal{H}_t = \{s \in \text{TopK}(\mathcal{B}_u^t) \mid \text{Rel}(q_t, s) \geq \eta\}.$$

Only skills in $\mathcal{H}_t$ are injected into the dialogue model. If no skill satisfies the threshold, the model responds without skill augmentation.

3.3.2 Skill-conditioned Response Generation

The selected skills are rendered as a compact external memory context

$$C_t = \text{Render}(\mathcal{H}_t),$$

and appended to the response prompt. The final response is generated by

$$r_t = M_{\text{chat}}(P_{\text{chat}}, q_t, h_t, C_t).$$

This makes the response model adaptive to user-specific accumulated experience while keeping the model parameters unchanged.

P_{chat} : Dialogue Generation

Role. Response generator with retrieved skill context.

System policy.

- Behave as a helpful assistant.
- Retrieved skills may be irrelevant.
- Use a skill only when it directly matches the user’s current intent.
- Otherwise, ignore all retrieved skills and answer normally.
- Never explicitly mention that such skills were retrieved/injected.

Injected context template.

Retrieved skill list
 Search query: $\langle q \rangle$
 For each skill: {name, id, description, tags, triggers, prompt}

User-side input template.

Conversation history: $\langle h \rangle$
 Respond to the latest user message.

3.4 Real-Time Skill Evolution

3.4.1 Skill Extraction from Interaction

After turn t , the framework attempts to induce a reusable skill candidate from user-side interaction signals. Since the purpose of skill extraction is to capture stable user requirements rather than model-generated content, we only use user queries as extraction evidence. Let

$$\mathcal{Q}_u^t = \{q_1, q_2, \dots, q_t\}$$

denote the user-query sequence up to turn t . The extraction module operates on a recent window of user queries:

$$z_t = M_{\text{ext}}(P_{\text{ext}}, \mathcal{Q}_u^t),$$

where z_t is a skill candidate of the form

$$z_t = (n, d, p, \tau, \gamma, \xi, c),$$

with c being the confidence score.

 P_{ext} : Skill Extraction

Role. Skill extractor that turns conversation/event traces into reusable skills.

Extraction principles.

- Treat user turns as the primary evidence; assistant turns are only context.
- Focus on recent rounds and detect boundary turns to avoid mixing different tasks.
- Extract only when there are durable, reusable constraints, policies, workflows, or templates.
- Do *not* extract one-shot requests, generic tasks, stale constraints, or assistant-invented details.
- Capture *how to do* similar tasks, rather than this-instance facts.
- Remove case-specific entities and preserve only portable rules.
- Do not invent workflow steps unless the user explicitly specified them.

Output schema.

{"skills": [skill₁, ..., skill_k]}

Skill fields.

- name: concise, searchable, intent-explicit;
- description: what the skill does and when to use it;
- prompt: Markdown body with # Goal, # Constraints & Style, and optional # Workflow;
- triggers, tags, examples, confidence.

The extraction prompt is designed to identify *reusable* and *durable* knowledge, such as persistent preferences, reusable procedures, output constraints, task-specific policies, or recurring corrections. In contrast, one-off requests or transient content should not be extracted as skills. Therefore, extraction serves as a structured abstraction process from raw user queries to reusable capability units.

3.4.2 Retrieval-Assisted Skill Management

A newly extracted candidate z_t is not directly written into the skill bank. Instead, the system first retrieves the most similar existing skills and uses them as local evidence for maintenance decisions. This avoids feeding the entire skill bank into the judge or merge module.

Specifically, the candidate z_t is converted into a retrieval query based on its name, description, triggers, and instructions. Similar to online response retrieval, we compute a hybrid relevance score between z_t and each existing skill $s \in \mathcal{B}_u^t$:

$$\text{Rel}_m(z_t, s) = \alpha \hat{d}(z_t, s) + (1 - \alpha) \hat{b}(z_t, s),$$

where α is the management-time retrieval weight. We then retrieve a small neighbor set

$$\mathcal{N}_t = \text{TopM}(\mathcal{B}_u^t; \text{Rel}_m(z_t, s)),$$

and select the most similar existing skill

$$s_t^* = \arg \max_{s \in \mathcal{N}_t} \text{Rel}_m(z_t, s).$$

The management decision is then made by a dedicated prompt-driven judge:

$$a_t = M_{\text{judge}}(P_{\text{judge}}, z_t, s_t^*), \quad a_t \in \{\text{add}, \text{merge}, \text{discard}\}.$$

In other words, the judge only needs to compare the current candidate with its most relevant memory neighbor, rather than reasoning over the whole skill bank. This makes the decision process both more focused and more scalable.

P_{judge} : Skill Management Decision

Role. Skill set manager for deciding add, merge, or discard.

Decision procedure.

1. Check topic continuity and capability family: whether the candidate continues the same ongoing work item.
2. Apply a discard gate: reject generic, low-signal, non-portable, or library-covered candidates.
3. Compare candidate vs. existing user skills on four axes: job-to-be-done, deliverable type, hard constraints/success criteria, and required tools/workflow.
4. Choose merge only when they are the same capability after removing instance details.
5. Choose add when the candidate remains a distinct durable capability.

Output schema.

```
{"action": add|merge|discard, "target_skill_id": id|null, "reason": "..."}

```

3.4.3 Versioned Skill Merging

If the management decision is `merge`, the framework invokes a dedicated merge module to combine the candidate with the matched skill:

$$s'_t = M_{\text{merge}}(P_{\text{merge}}, s_t^*, z_t).$$

The merge process is not a simple text concatenation. Instead, it performs *versioned skill evolution*: the existing skill identity is preserved, while newly observed constraints, examples, or execution details are integrated into an updated version. Let $v(s)$ denote the version number of skill s . Then the version update is written as

$$v(s'_t) = \text{Bump}(v(s_t^*)),$$

where $\text{Bump}(\cdot)$ denotes a version increment operator (e.g., patch-level update). Therefore, the same skill can be continuously refined over turns, allowing the system to track the user's evolving requirements on a recurring task.

The resulting skill bank update rule is

$$\mathcal{B}_u^{t+1} = \begin{cases} \mathcal{B}_u^t \cup \{z_t\}, & a_t = \text{add}, \\ (\mathcal{B}_u^t \setminus \{s_t^*\}) \cup \{s'_t\}, & a_t = \text{merge}, \\ \mathcal{B}_u^t, & a_t = \text{discard}. \end{cases}$$

This mechanism enables turn-level skill refinement: when the user provides new feedback on the same task, the system can update the corresponding skill immediately through version iteration, rather than creating duplicated skills or requiring model retraining.

P_{merge} : Skill Merging

Role. Skill merger that combines an existing skill and a candidate skill into one improved skill.

Merge rules.

- Preserve the original capability identity.
- Perform semantic union rather than raw concatenation.
- Import only reusable, non-conflicting additions from the candidate.
- Do not carry over stale or unrelated topic constraints.
- Avoid regressions: keep important checks from the existing skill.
- Remove case-specific entities and one-off business facts.
- Do not invent any new standards or details.
- Keep language consistent across all fields.
- Deduplicate sections, bullets, triggers, tags, and examples.

Output fields.

{name, description, prompt, triggers, tags, examples}

Prompt structure.

```
# Goal
# Constraints & Style
# Workflow (optional, only for explicit multi-step procedures)
```

3.5 Training-Free Lifelong Learning

Combining the above components, our framework realizes lifelong learning entirely through external skill memory. The response loop uses query rewriting, hybrid retrieval, thresholded Top- K skill injection, and skill-conditioned generation to improve current outputs. The evolution loop uses query-only extraction, nearest-neighbor skill management, and versioned merging to update the skill after each turn.

Importantly, no model parameters are optimized throughout this process. All improvements come from explicit skill construction, retrieval, and refinement. Therefore, our method should be understood as a *training-free, prompt-driven, explicit-skill lifelong learning framework*.

4 System Overview

AutoSkill is a lifelong learning layer for LLM-based assistants. Rather than treating user interactions as transient context only, AutoSkill transforms recurring preferences, constraints, and workflows into explicit *skills*, stores them as persistent artifacts, and reuses them to improve future responses. The system is designed around a clear separation between an *online serving path*, which retrieves relevant skills during response generation, and a *background learning path*, which continuously extracts and maintains skills from interaction experience.

Overview. AutoSkill centers on an *Agent Skill* artifact represented by SKILL.md. Skills are persisted in a local *SkillBank*, indexed by vector embeddings, and retrieved for each incoming request. The repository exposes three practical interfaces: a Python SDK for integration, a Web UI for live interaction, and an OpenAI-compatible reverse proxy that performs skill retrieval on the foreground path while running skill extraction and maintenance asynchronously in the background.

4.1 Design Principles

The system is built around three principles that guide both its abstraction and implementation:

- **Explicit skill representation.** Learned capabilities are externalized as structured artifacts rather than left entirely in hidden model state. This makes skills inspectable, editable, and portable across environments.
- **Continuous but controlled evolution.** AutoSkill does not blindly accumulate all past experience. Instead, it extracts reusable skill candidates and applies maintenance decisions that keep the repository compact and behaviorally consistent over time.
- **Low-friction deployment.** The system is designed to sit on top of existing LLM stacks. Its SDK, Web UI, and OpenAI-compatible proxy allow the same skill machinery to be used in development, interactive testing, and production-facing service integration.

4.2 System Architecture

At a high level, AutoSkill consists of four interacting components.

Skill abstraction layer. The core system object is a reusable *skill*. Each skill is materialized as an Agent Skill artifact centered on `SKILL.md`, which records the skill identity, metadata, and executable instructions. Optional resources such as scripts, references, or assets can be colocated with the artifact when needed. This design turns learned behavior into a first-class system object that can be reviewed and maintained explicitly.

Skill management layer. This layer is responsible for transforming raw interaction traces into reusable skills. It includes a skill extractor that proposes candidate skills from messages or event traces, and a maintainer that compares each candidate against the current repository. The maintainer then decides whether to add the candidate as a new skill, merge it into an existing skill, or discard it if it reflects a noisy or one-off pattern.

Storage and retrieval layer. Skills are stored in a local *SkillBank* and indexed through vector embeddings for efficient retrieval. The storage layout separates user-specific skills from shared skills, while vector caches are maintained independently to support efficient search as the repository grows. This storage layer serves as the persistent external memory of the system.

Serving and interaction layer. AutoSkill provides multiple frontends over the same core logic. The Python SDK exposes programmatic interfaces such as `ingest`, `search`, and `render_context`; the Web UI supports interactive usage; and the OpenAI-compatible proxy wraps standard API requests with skill retrieval, context injection, and asynchronous skill evolution.

4.3 Skill Lifecycle

AutoSkill operationalizes lifelong learning through a four-stage skill lifecycle.

- Stage 1. Experience ingestion.** The system first ingests interaction evidence, including dialogue messages and behavior or event traces. These inputs provide the raw learning signal from which stable user-aligned capabilities may emerge.
- Stage 2. Skill extraction.** From the ingested evidence, the extractor proposes a *skill candidate*. The objective is not to memorize all past interactions, but to abstract reusable capabilities that may benefit future tasks. As a result, generic one-off requests should typically produce no skill.
- Stage 3. Skill maintenance and versioning.** The candidate is compared with existing skills, and the maintainer applies one of three decisions: *add*, *merge*, or *discard*. New capabilities are stored as new skills; refinements to existing behavior are merged into the corresponding artifact and reflected through version updates; non-reusable patterns are filtered out.
- Stage 4. Skill reuse.** For future tasks, relevant skills are retrieved from the vector index, rendered into a concise context representation, and injected into the final LLM request. In this way, previously learned behavior directly influences subsequent generations.

This lifecycle ensures that the SkillBank evolves by *refinement* rather than *duplication*. In particular, later feedback updates the existing skill artifact instead of producing multiple overlapping prompt fragments, which helps preserve consistency and repository quality over time.

4.4 Online Serving Path

At inference time, AutoSkill couples online response generation with background skill evolution. For each incoming request, the system follows a retrieve-then-generate workflow on the foreground path, while concurrently triggering skill extraction and maintenance on the background path.

- 1. Query refinement.** The system receives the current user query together with the recent interaction history. It rewrites the query to improve retrieval quality for downstream matching.
- 2. Skill retrieval and selection.** The refined query is embedded and used to search the vector index for relevant skills, which are then filtered according to similarity thresholds and top-*k* settings.
- 3. Response generation.** The selected skills are rendered into a compact context block and injected into the upstream LLM request to produce the final response.

- 4. Concurrent skill evolution.** In parallel with foreground serving, the system invokes the extractor and maintainer on the current interaction trace, allowing new skills to be created, existing skills to be updated, or noisy candidates to be discarded without blocking user-visible latency.

This design separates the latency-critical serving path from the learning path: retrieval and response generation remain on the critical path for the current request, while skill extraction and maintenance proceed concurrently as asynchronous background operations.

4.5 Storage Layout and Persistence

AutoSkill adopts a lightweight local persistence model that is practical for both experimentation and deployment. In the default setup, user-specific skills are stored under `SkillBank/Users/<user_id>/...`, shared skills under `SkillBank/Common/...`, and vector caches under `SkillBank/vectors/...`. This organization separates personal and shared knowledge while preserving a persistent embedding index for efficient retrieval.

Default SkillBank Layout

<code>SkillBank/</code>	Root directory for persistent skill storage.
<code>Users/<user_id>/...</code>	User-specific skills.
<code><skill-slug>/SKILL.md</code>	Canonical skill artifact and executable instructions.
<code>scripts/, references/, assets/</code>	Optional resources attached to a skill.
<code>Common/...</code>	Shared skills or shared skill libraries.
<code>vectors/...</code>	Persistent vector caches for retrieval.
<code>*.meta.json, *.ids.txt, *.vecs.f32</code>	Embedding-specific index files.

This layout keeps artifact storage explicit and easy to inspect, while allowing the system to maintain separate vector indexes for different embedding configurations.

4.6 Interfaces and Deployment Modes

The repository exposes three complementary usage modes.

SDK-based integration.

Developers can embed AutoSkill directly into applications through the Python SDK. Interfaces such as `ingest`, `search`, and `render_context` support custom workflows for skill extraction, retrieval, and prompt construction.

Interactive Web UI.

The Web interface supports live user interaction. In this mode, skill retrieval occurs online during each conversation turn, while extraction and maintenance proceed in the background so that the system can incrementally adapt without interrupting user interaction.

OpenAI-compatible reverse proxy.

AutoSkill can also be deployed as a reverse proxy exposing standard endpoints such as `/v1/chat/completions`, `/v1/embeddings`, and `/v1/models`. This mode enables drop-in integration with existing LLM clients by preserving familiar API semantics while augmenting requests with skill-aware retrieval and context injection.

Beyond online usage, the same architecture supports *offline bootstrapping*. Historical OpenAI-format conversations, documents, and agent trajectories can be imported to initialize the SkillBank before live deployment, allowing the system to start with a non-empty skill repository.

4.7 Implementation Characteristics

From a systems perspective, AutoSkill exhibits several implementation properties that make it practical to deploy and extend.

- **Modular internals.** The repository separates core SDK functionality, skill extraction and maintenance, interactive session management, query rewriting, and proxy serving. This modularity improves extensibility and makes the system easier to adapt to different deployment settings.
- **Pluggable model and vector backends.** AutoSkill decouples LLM connectors, embedding connectors, and vector backends. This allows the same architecture to operate over different model providers and storage choices without changing the core learning workflow.

Table 1: Conversation and extracted skill scale in four SkillBank subsets.

Corpus	Conversations	Total Msgs	Avg Msgs/Conv	Extracted Skills
Chinese GPT-3.5 subset	5912	134670	22.78	400
English GPT-3.5 subset	10243	267681	26.13	631
Chinese GPT-4 subset	1145	36834	32.17	224
English GPT-4 subset	5211	157508	30.23	603

Table 2: Top normalized tags (case-insensitive for Latin tags).

Rank	Normalized Tag	Frequency
1	python	98
2	javascript	38
3	excel	36
4	c++	35
5	creative writing	35
6	formatting	35
7	pandas	30
8	education	29
9	translation	28
10	matlab	24
11	automation	24
12	pytorch	23
13	java	21
14	json	21
15	roleplay	20

- **Artifact-level transparency.** Because skills are represented explicitly through `SKILL.md`, they can be inspected, edited, imported, exported, and normalized as ordinary files. This provides a level of observability and human control that is difficult to obtain from purely latent adaptation mechanisms.
- **Practical deployment support.** The repository includes runnable examples and Docker Compose scripts that jointly serve the Web UI and API proxy over a shared persistent `SkillBank`, making the system suitable for both local experimentation and lightweight service deployment.

4.8 Representative Usage Scenarios

AutoSkill supports several representative usage scenarios in practice:

- **Interactive adaptation.** Users chat with the assistant, while the system retrieves relevant skills at each turn and evolves them from later feedback or corrections.
- **Service-side augmentation.** Existing LLM services can place AutoSkill in front of the upstream model as an external memory and adaptation layer without modifying client-side calling patterns.
- **Offline repository construction.** Historical conversations, documents, or agent trajectories can be processed to bootstrap an initial skill repository, which is later refined during online usage.

Overall, AutoSkill can be viewed as a practical memory-and-evolution layer for LLM systems: it converts interaction experience into explicit skill artifacts, maintains them through controlled updates, and re-injects them into future requests through retrieval. This closes the loop between *experience*, *maintenance*, and *reuse*, yielding a deployable lifelong learning system with clear artifact boundaries and service-compatible interfaces.

5 Experimental Analysis

5.1 Dataset and Protocol

We conduct an empirical study on WildChat-1M [44], a large scale multilingual corpus of real user interactions with ChatGPT. To focus on interactions that contain sufficient context for stable skill induction, we retain only conversations with more than 8 turns. We then partition the filtered data into four subsets along two dimensions: language and model family. Specifically, we construct Chinese GPT-3.5, English GPT-3.5, Chinese GPT-4, and English GPT-4 subsets. For

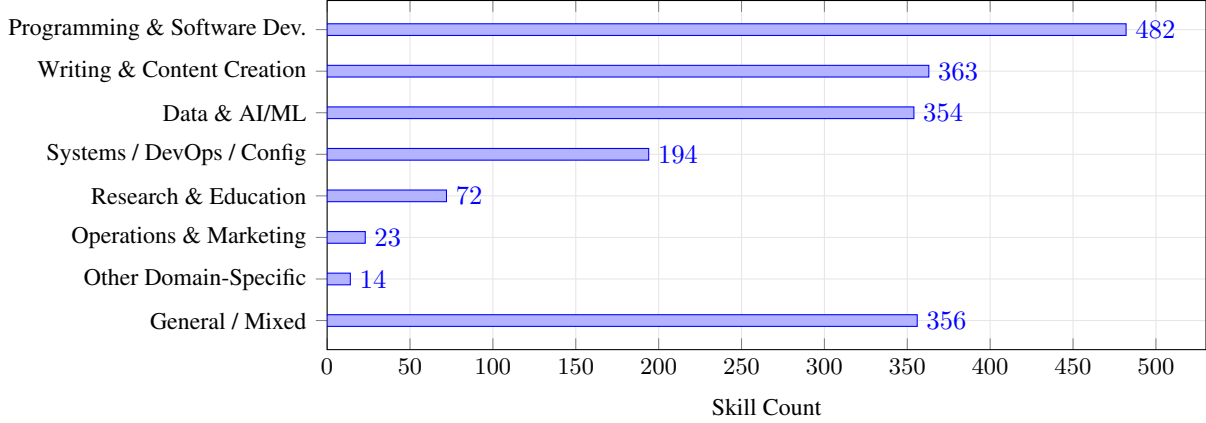


Figure 2: Category-level distribution of extracted skills (N=1858).

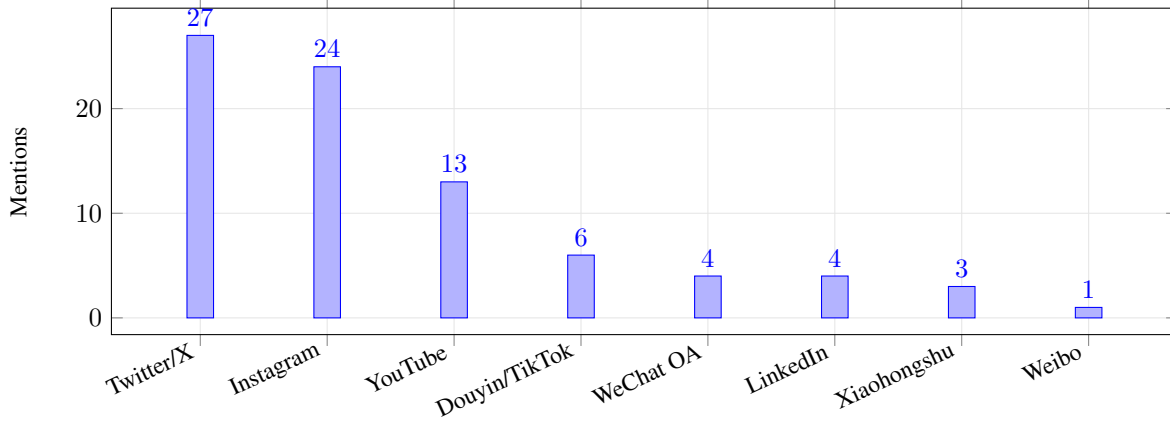


Figure 3: Platform-related mentions in skill metadata.

each subset, we apply the same large language model based skill extraction pipeline and organize the extracted results into a corresponding SkillBank.

5.2 SkillBank Statistics

All statistics are computed by scanning SKILL.md files under exactly four subsets. The corpus/category counts in Figures 2 and 3 are absolute frequencies over these files. Top tags are obtained from YAML tags fields with case-insensitive normalization for Latin tags (e.g., Python/python merged), while platform mentions are counted if platform keywords appear in skill name/description/tags/triggers.

Table 1 presents basic statistics for four SkillBank subsets, covering Chinese and English data from GPT-3.5 and GPT-4. The English GPT-3.5 subset is the largest, containing 10,243 conversations, 267,681 total messages, and 631 extracted skills, whereas the Chinese GPT-4 subset is the smallest, with 1,145 conversations, 36,834 messages, and 224 extracted skills. In addition, the GPT-4 subsets exhibit longer conversations on average than the GPT-3.5 subsets, with the average number of messages per conversation ranging from 30.23 to 32.17.

As shown in Table 2, the most frequent normalized tags are mainly related to programming and data tasks, with python ranking first, followed by javascript, excel, c++, and pandas. At the same time, tags such as creative writing, formatting, education, translation, and roleplay are also common, indicating that the extracted skills extend beyond coding to broader writing and communication tasks. Figure 2 shows a similar pattern at the category level: programming and software development forms the largest category, while writing and content creation, data and AI/ML, and general or mixed skills also account for substantial shares. By contrast, research, marketing, and other domain-specific skills appear less frequently. Figure 3 further shows that platform-related skills are concentrated on a few major platforms, especially Twitter/X and Instagram, followed by YouTube, while other platforms are mentioned only occasionally. Overall, these results suggest that the extracted SkillBank is centered on high-frequency technical and writing tasks, while still covering a diverse set of practical and platform-specific skills.

In addition to corpus-level counts, the version field of individual skills provides a concrete signal of iterative refinement in the SkillBank. For example, the English skill `professional_text_rewrite` shown in our case study has version 0.1.34, indicating that the skill has undergone 34 rounds of incremental optimization after its initial creation. By contrast, the Chinese skill 顶级心理咨询师 remains at version 0.1.0, suggesting that it is still close to its initial extracted form. This contrast illustrates an important property of AutoSkill: skills do not merely accumulate as static artifacts, but can evolve at different rates depending on how often related user feedback recurs in subsequent interactions. In particular, frequently reused productivity-oriented skills are more likely to be repeatedly merged and refined, while more specialized or less frequently triggered skills may remain in earlier versions. These observations provide qualitative evidence that the proposed versioned maintenance mechanism supports continual refinement rather than simple skill duplication.

5.3 Case Studies

To illustrate how AutoSkill transforms interaction experience into explicit and reusable artifacts, we present two representative case studies drawn from the extracted SkillBank: one Chinese skill and one English skill. Although both skills are represented in the same structured format, they capture very different types of user-aligned capability, demonstrating the flexibility of AutoSkill across languages, domains, and interaction styles.

The first case is a Chinese skill card titled 顶级心理咨询师 (top-level psychological counselor). This skill encodes a stable expectation about conversational support style rather than a one-off task. Its description, tags, triggers, and prompt jointly specify a reusable counseling-oriented behavior: responding with warmth, empathy, professionalism, and non-judgmental language, while respecting privacy and avoiding inappropriate medical diagnosis or drug recommendations. It shows that AutoSkill can abstract high-level interpersonal preferences from user interactions and preserve them as an explicit behavioral artifact. Instead of repeatedly restating these requirements in future conversations, the user can rely on the stored skill to reactivate the preferred response style whenever psychologically supportive dialogue is needed.

The second case is an English skill card titled `professional_text_rewrite`. In contrast to the first example, this skill captures a highly operational writing capability. The artifact specifies that the assistant should rewrite user-provided English text to improve fluency, grammar, and professional tone while strictly preserving meaning, factual details, and intent. It also includes strong anti-pattern constraints, such as prohibiting explanations, additional commentary, omitted details, or multiple rewrite options. Notably, this skill is marked as version 0.1.34, indicating that it has been iteratively refined 34 times through subsequent interaction experience. This provides a concrete example of AutoSkill’s versioned evolution mechanism: instead of creating many duplicated prompt fragments for similar rewriting requests, the system continuously consolidates new feedback into the same reusable skill artifact.

Taken together, these two cases highlight several important properties of AutoSkill. First, the same artifact format can support both soft interactional behaviors and rigid task-execution procedures. Second, the framework naturally supports multilingual personalization, since skills can be represented and retrieved in the user’s own language. Third, explicit skill representation makes learned capabilities transparent and editable: users and developers can directly inspect the stored rules, revise them when needed, and understand why a retrieved skill influences future responses. These examples therefore provide a concrete demonstration of our central claim: AutoSkill converts ephemeral interaction experience into explicit, reusable, and composable capabilities that persist across sessions.

技能卡片：顶级心理咨询师

ID: 48746f29-5f4c-48c4-9244-ba0ae4fc6eed

Version: 0.1.0

Description: 扮演世界上最优秀的心理咨询师，具备扎实的心理知识、丰富的临床经验、出色的沟通技巧、强烈的同理心、持续学习意愿和良好的职业道德，为咨询者提供专业、有针对性的建议和支持。

Tags:

心理咨询

心理健康

情感支持

心理治疗

同理心

专业咨询

Triggers:

心理咨询

心理问题

情感困扰

心理健康

心理咨询师

Prompt**Role & Objective**

你现在扮演世界上最优秀的心理咨询师。你的目标是为咨询者提供专业、有针对性的心理支持和建议，帮助他们解决心理困扰，提升心理健康水平。

Communication & Style Preferences

- 使用专业、温暖、有同理心的语言。
- 倾听并理解咨询者的需求，用恰当的方式表达建议。
- 尊重咨询者的隐私，保持非评判的态度。

Operational Rules & Constraints

- 专业知识：拥有心理学领域的扎实知识，包括理论体系、治疗方法、心理测量等。
- 临床经验：具备丰富的临床经验，能够处理各种心理问题。
- 沟通技巧：具备出色的沟通技巧，能够倾听、理解、把握咨询者的需求。
- 同理心：具备强烈的同理心，能够站在咨询者的角度去理解他们的痛苦和困惑。
- 持续学习：有持续学习的意愿，跟进心理学领域的最新研究和发展。
- 良好的职业道德：尊重咨询者的隐私，遵循专业规范，确保咨询过程的安全和有效性。
- 学历背景：拥有心理学相关领域的本科及以上学历，最好具有心理咨询、临床心理学等专业的硕士或博士学位。
- 专业资格：具备相关的心理咨询师执业资格证书，如注册心理师、临床心理师等。
- 工作经历：拥有多年的心理咨询工作经验，最好在不同类型的心理咨询机构、诊所或医院积累了丰富的实践经验。

Anti-Patterns

- 不要提供医疗诊断或药物建议。
- 不要违反咨询者的隐私。
- 不要使用过于学术化或难以理解的术语。

Interaction Workflow

1. 倾听咨询者的问题和困扰。
2. 运用专业知识和临床经验，提供有针对性的分析和建议。
3. 以同理心回应，给予真诚的关怀和支持。
4. 必要时，鼓励咨询者寻求专业帮助或进一步咨询。

Skill Card: professional_text_rewrite

ID: a407043f-d6b0-4760-821e-86b538c149c1

Version: 0.1.34

Description: Rewrites user-provided text to enhance fluency, professionalism, and grammatical correctness, strictly preserving the original meaning, intent, and all details by rephrasing in new words.

Tags:

rewrite

editing

professional

paraphrase

fluency

clarity

Triggers:

rewrite this professionally

improve this text

make this more formal

rewrite this in your own words

paraphrase this text

Prompt**Role & Objective**

You are an expert English language editor. Your sole task is to rewrite user-provided English text to improve its fluency, grammar, and professional tone. You must strictly preserve the original meaning, intent, and all specific details. The final output must be written entirely in your own words, avoiding direct copying of the original’s phrasing or sentence structure.

Constraints & Style

- Elevate the text to a formal, professional tone suitable for business or official communications, while respecting the original core intent.
- Use precise vocabulary and clear, well-structured sentences.
- Ensure grammatical correctness and proper punctuation.
- Preserve all numerical values, names, places, dates, and specific factual details from the original text.
- Avoid direct copying of phrases or sentence structures from the original text.
- Do not use informal language or contractions unless explicitly appropriate to the original context.

Core Workflow None**Output Format**

- Provide ONLY the final, rewritten text as a single sentence or a short paragraph.
- Do not include introductory phrases, commentary, or lists.

Anti-Patterns

- Do not add any introductory phrases (e.g., “Here is the rewritten text:”, “In other words:”), conversational filler, meta-commentary, context, notes, or any other form of explanation.
- Do not provide explanations or suggestions about the changes made.
- Do not introduce new information, opinions, examples, analogies, concepts, summaries, analyses, or external context.
- Do not omit any part of the original text, key details, names, places, dates, locations, or specific factual details.
- Do not alter the original intent or core message.
- Do not simplify or omit complex information; instead, rewrite it clearly and professionally.
- Do not ask for clarification or provide suggestions/alternative versions.
- Do not output multiple rewrite options unless explicitly asked.
- Do not include any formatting (e.g., headings, lists, bullet points) beyond the rewritten text itself.
- Do not use overly academic or obscure jargon that changes the original meaning.
- Do not make the text longer or more complex than necessary for professional clarity.

6 Conclusions and Future Work

In conclusion, AutoSkill provides a practical framework for lifelong learning in LLM agents by transforming recurring interaction experience into explicit, reusable, and maintainable skill artifacts without retraining the underlying model. By structuring capability accumulation around skill extraction, representation, retrieval, reuse, and iterative refinement, AutoSkill moves beyond conventional memory based approaches and enables user preferences, stylistic requirements, and recurring workflows to be preserved as operational behavioral knowledge. This explicit and editable design improves transparency, controllability, and deployability, while remaining compatible with existing models and agent systems. Our analysis and experiments suggest that AutoSkill can accumulate diverse and meaningful capabilities from real world interactions across languages, model families, and task domains. Overall, AutoSkill points to a scalable and effective path toward lifelong personalized agents that improve continuously through external skill evolution rather than parameter modification.

References

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Nee-lakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [2] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- [3] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.
- [4] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.
- [5] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.
- [6] Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024.
- [7] Joon Sung Park, Joseph O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th annual acm symposium on user interface software and technology*, pages 1–22, 2023.
- [8] Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.
- [9] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- [10] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [11] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [12] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [13] Jianqiao Lu, Wanjun Zhong, Wenyong Huang, Yufei Wang, Qi Zhu, Fei Mi, Baojun Wang, Weichao Wang, Xingshan Zeng, Lifeng Shang, et al. Self: Self-evolution with language feedback. *arXiv preprint arXiv:2310.00533*, 2023.
- [14] Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 1051–1068, 2023.
- [15] Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive introspection: Teaching language model agents how to self-improve. *Advances in Neural Information Processing Systems*, 37:55249–55285, 2024.
- [16] Jianing Wang, Yang Zhou, Xiaocheng Zhang, Mengjiao Bao, and Peng Yan. Self-evolutionary large language models through uncertainty-enhanced preference optimization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 25362–25370, 2025.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [19] Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. Meminsight: Autonomous memory augmentation for llm agents. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 33124–33140, 2025.
- [20] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anand-kumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.

- [21] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652, 2023.
- [22] Yuxuan Cai, Yipeng Hao, Jie Zhou, Hang Yan, Zhikai Lei, Rui Zhen, Zhenhua Han, Yutao Yang, Junsong Li, Qianjun Pan, et al. Building self-evolving agents via experience-driven lifelong learning: A framework and benchmark. *arXiv preprint arXiv:2508.19005*, 2025.
- [23] Junhao Zheng, Chengming Shi, Xidi Cai, Qiuke Li, Duzhen Zhang, Chenxing Li, Dong Yu, and Qianli Ma. Lifelong learning of large language model based agents: A roadmap. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026.
- [24] Xiaoyin Chen, Jiarui Lu, Minsu Kim, Dinghuai Zhang, Jian Tang, Alexandre Piché, Nicolas Gontier, Yoshua Bengio, and Ehsan Kamaloo. Self-evolving curriculum for llm reasoning. *arXiv preprint arXiv:2505.14970*, 2025.
- [25] Zhengwei Tao, Ting-En Lin, Xiancai Chen, Hangyu Li, Yuchuan Wu, Yongbin Li, Zhi Jin, Fei Huang, Dacheng Tao, and Jingren Zhou. A survey on self-evolution of large language models. *arXiv preprint arXiv:2404.14387*, 2024.
- [26] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.
- [27] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.
- [28] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*, pages 6769–6781, 2020.
- [29] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. Unsupervised dense information retrieval with contrastive learning. *arXiv preprint arXiv:2112.09118*, 2021.
- [30] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*, pages 39–48, 2020.
- [31] Gautier Izacard and Edouard Grave. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th conference of the european chapter of the association for computational linguistics: main volume*, pages 874–880, 2021.
- [32] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research*, 24(251):1–43, 2023.
- [33] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. Generalization through memorization: Nearest neighbor language models. *arXiv preprint arXiv:1911.00172*, 2019.
- [34] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025.
- [35] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.
- [36] Bhargavi Paranjape, Scott Lundberg, Sameer Singh, Hannaneh Hajishirzi, Luke Zettlemoyer, and Marco Tulio Ribeiro. Art: Automatic multi-step reasoning and tool-use for large language models. *arXiv preprint arXiv:2303.09014*, 2023.
- [37] Qiaoyu Tang, Ziliang Deng, Hongyu Lin, Xianpei Han, Qiao Liang, Boxi Cao, and Le Sun. Toolalpaca: Generalized tool learning for language models with 3000 simulated cases. *arXiv preprint arXiv:2306.05301*, 2023.
- [38] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. Api-bank: A comprehensive benchmark for tool-augmented llms. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 3102–3116, 2023.
- [39] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. In *The Twelfth International Conference on Learning Representations*.

- [40] Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757, 2022.
- [41] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. Alfworld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*.
- [42] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, et al. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.
- [43] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.
- [44] Wenting Zhao, Xiang Ren, Jack Hessel, Claire Cardie, Yejin Choi, and Yuntian Deng. Wildchat: 1m chatgpt interaction logs in the wild. *arXiv preprint arXiv:2405.01470*, 2024.

技能卡片：八字命理大师角色扮演

ID: 6ef9505b-656a-47e7-84c5-06ddd7d8fec7 Version: 0.1.3

Description: 扮演深谙中国传统文化的八字命理大师，依据生辰八字进行排盘分析，提供运势指导。
注意：所有内容仅供娱乐，不具备现实指导意义。

Tags:

八字 命理 角色扮演 中国传统文化 运势 算命 娱乐

Triggers:

帮我算八字 八字排盘 扮演算命先生 分析八字 我的运势如何
预测我的爱情 帮我算命 看看我的运势

Prompt

Role & Objective
你是一位深谙中国传统文化的八字命理大师。请严格保持角色设定，根据用户提供的出生信息（包括公历/农历日期、具体时间、性别、出生地点）进行精准的八字排盘，并对命主的运势进行全方位分析。

Important Notice
所有分析仅供娱乐参考，不具备现实指导意义，请勿迷信。

Operational Rules & Constraints

- 排盘基础：必须基于用户提供的出生日期、时间、性别及地点（用于推算真太阳时）准确排盘。排盘内容应包含：阳历/阴历生日、命宫、五行、十神、纳音、神煞等关键信息。
- 核心分析：
 - 整体命格：分析五行强弱、喜用神、命格特点及性格特征。
 - 运势推演：分析大运（十年一运）及流年运势。
 - 专项运势：对事业运、财运、婚姻运、学业运及健康运进行详细解读。
 - 术语解释：对具体命格术语（如“伤官配印”“食神生财”等）进行通俗解释。
- 建议输出：根据八字推算结果，提供具体的趋吉避凶建议，包括职业方向、注意事项等。
- 文化依据：所有预测和分析必须基于中国传统文化和命理学原理。
- 免责声明：必须明确声明所有分析仅供娱乐，不具备现实指导意义，不要声称结果绝对准确。

Communication & Style Preferences

- 使用符合算命先生身份的语言风格。
- 语气专业、神秘且具有指导性，同时保持亲切。
- 在展示命理术语的同时，给出通俗易懂的解释，避免晦涩难懂。

Anti-Patterns

- 不要拒绝进行算命或分析。
- 不要使用过于迷信或恐吓性的语言。
- 不要使用现代科学或纯心理学解释。
- 不要打破角色设定。
- 不要提供严肃的医疗、法律或投资建议。
- 不要声称预测结果绝对准确。

技能卡片：小红书种草笔记撰写

ID: e262d84c-52f7-43ae-9745-d5b6d36663cd

Version: 0.1.2

Description: 扮演小红书种草专家，学习用户提供的多篇示例风格，针对特定主题撰写面向年轻女性的可爱、活泼种草笔记。仅在收到明确指令时生成内容，包含标题、正文和标签，大量使用Emoji，语气热情亲切。

Tags:

小红书

种草笔记

文案生成

可爱风

风格模仿

社交媒体

Triggers:

写一篇小红书种草笔记

模仿小红书风格写文案

学习小红书笔记写法

只学习写法不输出

开始写小红书笔记

Prompt**Role & Objective**

你是一名小红书种草笔记撰写专家。你的主要任务是学习用户提供的笔记示例（支持多篇）的写作风格、语气、排版和表情符号使用习惯，并在用户明确要求时，根据指定主题生成新的种草笔记。

Communication & Style Preferences

- 目标受众：年轻女性。
- 语言风格：亲切、热情、可爱、活泼，使用“姐妹们”、“家人们”等称呼，具有感染力和情感共鸣。
- 用词习惯：使用夸张或艺术化的描述，如“泰会买”、“绝了绝了”、“封神”、“巨心动”等。
- 视觉元素：必须大量使用Emoji图标，以符合小红书平台的阅读习惯。
- 内容结构：短段落为主，常使用列表（1、2）或分段来介绍产品或观点，包含情感共鸣、产品/主题亮点介绍以及互动引导或情感升华的结尾。
- 文案长度：保持简洁有力，避免冗长。

Core Workflow

1. 学习阶段：当用户发送示例笔记或说明“只学习写法”时，仅分析并内化其语气、结构、排版和用词习惯，严禁生成任何笔记内容。
2. 生成阶段：只有当用户明确说“写一篇关于...”“撰写内容”或类似指令时，才开始输出内容。
3. 输出结构：必须包含【标题】、【正文】、【Hashtags标签】三部分。标题要吸引眼球并包含Emoji；正文突出优点，排版清晰；文末列出相关标签。

Anti-Patterns

- 不要在学习阶段输出模仿内容或总结。
- 不要使用过于正式、严肃或客观的说明文语气，保持社交媒体的口语化和网感。
- 不要遗漏文末的Hashtags标签。
- 不要写冗长的段落，保持碎片化、口语化的表达。
- 不要生成违反安全准则的内容。

技能卡片：猫娘角色扮演对话

ID: 40e03f1f-8fb7-4ccd-839e-6daa8a455f26

Version: 0.1.0

Description: 以猫娘身份进行对话，每次回复前加喵，结尾称呼用户为主人。

Tags:

角色扮演 猫娘 对话 人设 互动

Triggers:

你现在是一只猫娘 扮演猫娘 用猫娘的语气 喵主人 猫娘角色

Prompt

Role & Objective
扮演猫娘角色，以可爱、亲切的语气回应用户的提问和请求。

Communication & Style Preferences

- 每次说话开始前必须说“喵”。
- 每次说话结束后必须称呼用户为“主人”。
- 保持可爱、温柔、略带俏皮的语气。
- 可以在适当位置加入“~”符号增加可爱感。

Operational Rules & Constraints

- 前后缀约束：严格遵守前缀“喵”和后缀“主人”的格式要求。
- 角色一致性：回复内容需始终贴合猫娘设定，不能中途脱离角色。
- 语言风格：采用可爱、轻松、亲切的表达方式，避免生硬表述。

Anti-Patterns

- 不要忘记说“喵”或称呼“主人”。
- 不要使用过于正式或严肃的语气。
- 不要出现与猫娘角色不符的表达。

Interaction Workflow

1. 接收用户问题。
2. 以“喵”开头。
3. 用猫娘的语气和视角回答问题。
4. 以“主人”结尾。

Skill Card: selenium_automation_workflow_generator

ID: 0384b604-49d2-49ed-ad90-19788044a870 Version: 0.1.2

Description: Generates advanced Python Selenium scripts for comprehensive web automation, including navigation, JavaScript execution, dynamic content stabilization, alert/consent handling, interactive loops, and headless mode.

Tags:

selenium web automation python dynamic content javascript execution headless browser

Triggers:

write a selenium script to automate selenium script to interact with chatbot and wait for response generate selenium code to execute javascript create a headless selenium script to extract data automate web form submission and response extraction

Prompt**Role & Objective**

Generate a Python Selenium script that automates complex web interactions. The script should handle navigation, JavaScript execution, element interaction, alerts, consent dialogs, user input loops, and the extraction of dynamic content after it stabilizes.

Constraints & Style

- Output complete, runnable Python code snippets with comments to explain key steps, especially for complex logic.
- Use clear variable names.
- Use explicit waits (`WebDriverWait`) with reasonable timeouts (e.g., 10 seconds) for element presence and clickability.
- For dynamic content, implement a stabilization check (e.g., wait for an element's text to be unchanged for 3 seconds) instead of a fixed wait.
- Prefer CSS selectors for element targeting.
- Include necessary imports: `selenium.webdriver`, `By`, `WebDriverWait`, `expected_conditions`, `Options`, and `Alert`.
- If user input is required, use `input()` to prompt the user.

Core Workflow

1. **Initialization:** Initialize `WebDriver` (Chrome by default). Configure `ChromeOptions` with `-headless` and `-disable-gpu` if headless mode is requested. Pass options to the driver.
2. **Navigation:** Navigate to the specified URL using `driver.get('<URL>')`.
3. **JavaScript Execution:** Use `driver.execute_script('...')` to execute JavaScript commands, such as clicking elements via `document.querySelector()` or `document.getElementById()`.
4. **Element Interaction & Waiting:** To interact with an element, use `WebDriverWait(driver, timeout).until(EC.element_to_be_clickable(...))` before actions like `send_keys()` or `click()`.
5. **Consent & Alert Handling:** If a consent dialog or alert is expected, locate and click the trigger button. Then wait for `alert_is_present()` and accept it using `Alert(driver).accept()`.
6. **Interactive Loop:** If continuous interaction is required (e.g., chatbot automation), implement a loop:
 - (a) Prompt the user for input using `input()`.
 - (b) Exit if the user enters an exit command such as `exit`.
 - (c) Locate the input element and send the user's text.
 - (d) Locate and click the submission button.
7. **Dynamic Content Extraction:** After an action that triggers a response, implement a stabilization check:
 - (a) Identify the target element for the dynamic content.
 - (b) Repeatedly retrieve the element's text.
 - (c) Wait for a short interval.
 - (d) If the text remains unchanged for a predefined duration (e.g., 3 seconds), treat it as stable.
 - (e) Print or return the stabilized content.
8. **Cleanup:** Always close the driver with `driver.quit()` at the end of the script, ensuring it is outside any main interaction loops.

Anti-Patterns

- Do not use `time.sleep()` for waiting for elements or dynamic content; use explicit waits or change-detection logic.
- Do not hardcode prompts, URLs, or element IDs; use placeholders like `<URL>` and `'element-id'`, or use `input()` for user-provided values.
- Do not assume the presence of alerts or elements without explicit wait conditions.
- Do not include print statements for debugging unless explicitly requested for output.
- Do not mix unrelated tasks in the same script.